

Article Platform

Containerization & Deployment

Technical Report — Docker Assignment

Mehdy Mokhtari

Internet Engineering — Computer Assignment 7

Repository: github.com/MahdyMokh7/article-platform

Table of Contents

Table of Contents	2
1. Introduction and Scope	3
1.1 Deliverables	3
2. Core Docker Concepts Applied	3
2.1 Image vs. Container	3
2.2 Docker vs. Virtual Machines	3
2.3 The Dockerfile and the Build Process	4
3. Frontend Containerization	4
3.1 Multi-Stage Build Strategy.....	4
3.2 Base Image Selection.....	4
3.3 Build-Time Environment Injection.....	5
4. Reverse Proxy	5
4.1 Concept.....	5
4.2 Application in This Project.....	5
4.3 Rationale and Trade-offs	6
5. Backend Containerization.....	6
5.1 Multi-Stage Build Strategy.....	6
5.2 Base Image Selection.....	7
5.3 Additional Hardening.....	7
6. Docker Compose Orchestration	7
6.1 Service Topology	7
6.2 Networking Model.....	7
6.3 Startup Ordering and Health Checks	8
7. Persistence: Docker Volumes	8
7.1 Named Volumes vs. Bind Mounts	8
7.2 Application to PostgreSQL.....	8
8. Database Containerization	9
9. Secrets and Configuration Security	9
10. Verification and Testing Performed	10
11. Conclusion	10

1. Introduction and Scope

This report documents the containerization phase of the Article Platform project: a Spring Boot REST API, a React single-page application, and a PostgreSQL database, brought together under Docker and Docker Compose. It is written to accompany the code submission and to serve as the reference material for the oral defense, covering the reasoning behind every design decision — not just what was built, but why it was built that way.

The scope matches the assignment specification exactly: Dockerize the backend, Dockerize the frontend using a multi-stage build with Nginx as a static file server and reverse proxy, orchestrate all three services (backend, frontend, database) with a single Docker Compose file, and apply the underlying concepts of images, containers, networking, volumes, and secret handling correctly rather than by accident.

1.1 Deliverables

File	Location	Purpose
Dockerfile	backend/	Multi-stage build: compile with Maven, run with a JRE-only image
.dockerignore	backend/	Excludes target/, IDE files, and logs from the build context
Dockerfile	frontend/	Multi-stage build: compile the Vite app, serve with Nginx
nginx.conf	frontend/	Static file serving + reverse proxy configuration for Nginx
.dockerignore	frontend/	Excludes node_modules/, dist/, and env files from the build context
docker-compose.yml	repository root	Orchestrates postgres, backend, and frontend as one stack
.env.example	repository root	Template for the secrets/config Compose expects at runtime

2. Core Docker Concepts Applied

2.1 Image vs. Container

An image is an immutable, read-only package containing an application together with everything it needs to run: a base operating system layer, a runtime, libraries, and the compiled application itself. A container is a running instance of an image — the same relationship a class has to an object. Multiple containers can be started from the same image, each with its own writable layer and process space, but sharing the identical underlying image layers on disk.

In this project, three images are produced: article-platform-backend, article-platform-frontend, and the official postgres:16-alpine image pulled as-is. docker-compose up then creates one container from each.

2.2 Docker vs. Virtual Machines

The assignment specifically asks for a comparison of Docker's advantages over a VM, since both isolate applications. The distinction is what layer the isolation happens at:

- A VM virtualizes hardware and runs a full guest operating system (its own kernel) on top of a hypervisor — heavyweight (gigabytes per VM), slow to boot (minutes), and the resources allocated to it are largely fixed.
- A container virtualizes at the operating-system level: all containers on a host share the host's kernel and are isolated from each other using kernel namespaces and cgroups. This makes containers lightweight (megabytes), fast to start (fractions of a second, as observed when starting the postgres container in testing), and cheap to run many of side by side.

For this project the practical benefit is concrete: the backend image final stage is under 300 MB and starts in seconds, whereas a VM shipping a full Linux distribution plus a JDK would be an order of magnitude larger and slower — for no additional isolation benefit relevant to a three-service web application.

2.3 The Dockerfile and the Build Process

A Dockerfile is a sequential script of build instructions. Each instruction produces a new, cached image layer; unchanged layers are reused on subsequent builds, which is why both this project's Dockerfiles copy dependency manifests (pom.xml, package.json) and resolve dependencies before copying the rest of the source code — so that editing a Java class or a React component does not force a full re-download of every dependency on the next build.

3. Frontend Containerization

3.1 Multi-Stage Build Strategy

The React application was previously run through the Vite development server (npm run dev), which is unsuitable for production: it serves unminified modules over a lightweight dev server with none of the caching, compression, or security headers a real deployment needs. The Dockerfile instead builds the app for production with npm run build, producing a small set of static HTML/CSS/JS files, and hands those files to Nginx — a purpose-built static file server.

The build uses two stages so that Node.js, npm, and the entire node_modules tree (over 100 MB) never appear in the final image:

```
FROM node:22-alpine AS build      # stage 1: has Node/npm, builds the static bundle
...
FROM nginx:1.27-alpine AS runtime # stage 2: only Nginx + the compiled dist/ folder
COPY --from=build /app/dist /usr/share/nginx/html
```

The final image contains only Nginx and static files — no JavaScript runtime, no package manager, no source code.

3.2 Base Image Selection

Stage	Image	Justification
Build	node:22-alpine	Matches the project's Node 18+ requirement; Alpine variant is used because the stage is discarded after the build, so its only cost is build time, which Alpine minimizes via faster layer pulls.
Runtime	nginx:1.27-alpine	Official, minimal, actively maintained image. Alpine base keeps the shipped image small (~40 MB) and reduces the OS package surface exposed to the public internet, since this container is the one directly reachable by end users.

3.3 Build-Time Environment Injection

The frontend reads its API base URL from `import.meta.env.VITE_API_BASE_URL` (see `frontend/src/services/axiosConfig.js`). Vite variables of this kind are not read at container runtime — they are statically inlined into the JavaScript bundle at build time. Passing this as a runtime environment variable to the container would therefore have no effect, since by the time the container starts, the bundle has already been compiled.

The Dockerfile instead accepts it as a build argument and writes it to a `.env.production` file that Vite consumes during the build step:

```
ARG VITE_API_BASE_URL=/api
RUN echo "VITE_API_BASE_URL=${VITE_API_BASE_URL}" > .env.production
RUN npm run build
```

The default value, `/api`, is a same-origin relative path rather than an absolute URL such as `http://backend:8080/api` — this is intentional and is explained in the reverse proxy section below.

4. Reverse Proxy

4.1 Concept

A forward proxy sits in front of clients and hides them from the servers they contact. A reverse proxy is the inverse: it sits in front of one or more servers and hides them from the client. The client only ever addresses the proxy; the proxy decides, transparently, which backend service actually handles each request.

4.2 Application in This Project

Nginx plays this role for the frontend container. It has two jobs configured in `nginx.conf`:

1. Serve the compiled React static files directly for any path that is not an API call, with a fallback to `index.html` so that React Router's client-side routes do not 404 on a hard refresh.
2. Forward every request under `/api/` to the backend service over the internal Docker network.

```
location /api/ {
    proxy_pass http://backend:8080/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
```

```

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location / {
    try_files $uri $uri/ /index.html;
}

```

Because backend resolves to the backend container only within the Compose network (see Section 6.2), this configuration is meaningful only once both containers are started together via Docker Compose.

4.3 Rationale and Trade-offs

Benefits realized in this project:

- Same-origin API calls: the browser addresses everything through `http://localhost:3000`, so requests to `/api/...` are same-origin, which eliminates CORS preflight requests in production entirely — the CORS allow-list in `SecurityConfig` is only exercised by the Vite dev server on port 5173, not by the containerized deployment.
- Single public entry point: only the frontend container needs to be reachable from outside the Docker network; the backend and database can, in a real deployment, remain unpublished and unreachable directly.
- Decoupling: the React source code never hardcodes the backend's host or port — that mapping lives in one place, `nginx.conf`, independent of how the frontend code itself is built.

Trade-offs accepted knowingly:

- An additional network hop is introduced per API call, though negligible on a local Docker bridge network.
- Nginx becomes a second point of failure: if it is down, both the UI and API access are unreachable even though the backend itself may be healthy.
- Debugging a failed request now requires checking two layers — Nginx's proxy logs and the backend's own logs — rather than one.

5. Backend Containerization

5.1 Multi-Stage Build Strategy

The backend follows the same two-stage principle as the frontend, but for a different reason: a JVM application does not need a Java compiler, Maven, or any build tooling to run — only a Java Runtime Environment and the already-compiled JAR file.

```

FROM maven:3.9.9-eclipse-temurin-22 AS build
COPY pom.xml .
RUN mvn -B dependency:go-offline          # cached layer: only reruns if pom.xml changes
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:22-jre-alpine AS runtime
COPY --from=build /app/target/*.jar app.jar

```

```
USER spring
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.2 Base Image Selection

Stage	Image	Justification
Build	maven:3.9.9-eclipse-temurin-22	Official Maven image bundled with an Eclipse Temurin JDK 22 — matches the project's <java.version>22 exactly, guaranteeing the bytecode produced is compatible with the runtime JRE below.
Runtime	eclipse-temurin:22-jre-alpine	JRE-only variant: no javac, no build tools, nothing beyond what is required to execute a jar. Combined with Alpine Linux, this keeps the shipped image small and reduces the OS package surface — directly following the assignment's instruction that the final backend image be JRE-based, not JDK-based.

5.3 Additional Hardening

- Non-root execution: a dedicated spring user is created and the application runs as that user rather than as root, limiting the blast radius if the container were ever compromised.
- Container healthcheck: the image defines a HEALTHCHECK against /v3/api-docs (exposed by the already-present springdoc-openapi dependency), used by Docker Compose to know when the backend is genuinely accepting requests rather than merely "running" — see Section 6.3. No new dependency (e.g. Spring Actuator) was introduced solely for this purpose.
- Dependency-layer caching: pom.xml is copied and resolved before the source tree, so incremental builds after a code change skip re-downloading the entire Maven dependency tree.
- Version-independent JAR copy: COPY --from=build /app/target/*.jar app.jar uses a wildcard rather than a hardcoded filename, since pom.xml does not set <finalName> — this keeps the Dockerfile correct even if the project version changes, without needing to edit pom.xml.

6. Docker Compose Orchestration

6.1 Service Topology

A single docker-compose.yml at the repository root defines three services — postgres, backend, and frontend — replacing three separate manual docker build / docker run invocations with one declarative file and one command, docker-compose up -d --build.

```
services:
  postgres:  image: postgres:16-alpine
  backend:   build: ./backend   (depends on postgres)
  frontend:  build: ./frontend   (depends on backend)
```

6.2 Networking Model

All three services are attached to a single custom bridge network, `article-platform-network`. Docker Compose automatically registers each container's service name as a DNS hostname within that network. This is what allows:

- `nginx.conf`'s `proxy_pass http://backend:8080/api/` to resolve to the backend container's internal IP address.
- The backend's `SPRING_DATASOURCE_URL, jdbc:postgresql://postgres:5432/${POSTGRES_DB}`, to resolve to the database container.

Containers on the same user-defined network can reach each other freely by name; only ports explicitly published with a `ports` mapping are reachable from the host machine. Deliberately, the `postgres` service publishes no port to the host — nothing outside the Docker network needs to reach it directly, since only the backend does, and it does so over the internal network. Only backend (8080) and frontend (80, mapped to the host's 3000) are published, matching the two URLs the README documents as the application's public entry points.

6.3 Startup Ordering and Health Checks

A plain `depends_on` only waits for a container to start, not for the service inside it to be ready — `Postgres`, for instance, reports its container as running well before it can accept connections. To avoid the backend attempting to connect before the database is genuinely ready, each service defines a healthcheck, and downstream services declare `condition: service_healthy` rather than a bare dependency:

Service	Healthcheck	Downstream dependency
<code>postgres</code>	<code>pg_isready -U ... -d ...</code>	backend waits for <code>service_healthy</code>
<code>backend</code>	<code>wget against /v3/api-docs</code>	frontend waits for <code>service_healthy</code>

This produces a deterministic startup order — database, then backend, then frontend — without relying on fixed sleep delays, which would be both slower and less reliable.

7. Persistence: Docker Volumes

7.1 Named Volumes vs. Bind Mounts

Containers are ephemeral by design: removing a container discards any data written to its writable layer. Docker offers two mechanisms to persist data outside that layer:

- A named volume is a storage area entirely managed by the Docker engine (typically under `/var/lib/docker/volumes` on the host). Docker controls its location and lifecycle; the container only sees a mount point.
- A bind mount maps a specific path on the host filesystem directly into the container. The host path is fully under the user's control, but the mapping is host-specific and less portable.

7.2 Application to PostgreSQL

This project uses a named volume for the database, declared once and mounted into the postgres service:

```
services:
  postgres:
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

A named volume was chosen over a bind mount because the data does not need to be inspected or edited directly from the host filesystem, and a named volume is portable across machines without adjusting host-specific paths — appropriate for a database whose data should survive docker-compose down but be fully removed by an explicit docker-compose down -v when a clean slate is wanted.

8. Database Containerization

Per the assignment's guidance, no custom Dockerfile was written for PostgreSQL — the official postgres:16-alpine image is used as-is, configured entirely through environment variables (POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD), which the base image's entrypoint script consumes on first startup to initialize the database, user, and credentials. This is the officially supported configuration mechanism for that image and avoids maintaining a redundant, custom database image.

A secondary effect of this setup, verified during testing, is that the backend requires zero code changes to run inside Docker: Spring Boot's environment-variable property source takes precedence over values defined in application.yml, so SPRING_DATASOURCE_URL / _USERNAME / _PASSWORD supplied by Compose transparently override the hardcoded local-development defaults already present in that file.

9. Secrets and Configuration Security

The assignment asks that sensitive configuration be protected "in the simplest way possible." The measures applied:

- No secret (database password, JWT signing key) is baked into any image layer. All of them are injected as environment variables at container-start time via docker-compose.yml, which itself reads from a local .env file.
- The real .env file is excluded from version control by the project's existing .gitignore; only .env.example — containing placeholder values and no real secrets — is committed, documenting exactly which variables a new environment must supply.
- application.yml's jwt.secret already used a \${JWT_SECRET:<fallback>} placeholder before this phase. That fallback is only ever reached if JWT_SECRET is left unset; in the Docker Compose flow it is always supplied explicitly from .env, so the checked-in fallback value is never actually used at runtime.
- The PostgreSQL container is not published to the host network, so it is unreachable from outside the Docker bridge network entirely — the credentials protecting it are a second layer of defense, not the only one.

- Both runtime images (backend and frontend) execute as non-root users rather than the container default of root.

10. Verification and Testing Performed

Each layer was built and validated independently before being combined, to isolate failures to a single cause:

3. Frontend image built standalone (`docker build ./frontend`) and run in isolation on port 3000, confirming the static SPA renders and client-side routing works.
4. Backend image built standalone (`docker build ./backend`) and run against a throwaway `postgres:16-alpine` container, using `--add-host` to simulate Compose's internal DNS resolution before `docker-compose.yml` existed; confirmed startup succeeds and `/swagger-ui.html` and `/v3/api-docs` respond.
5. Full reverse-proxy path exercised by running the frontend container together with a locally running (non-containerized) backend via `--add-host=backend:host-gateway`, confirming `/api/*` calls made from the browser at `localhost:3000` are correctly relayed.
6. Full stack brought up together with `docker-compose up -d --build`, confirming all three services report healthy and the application is reachable end-to-end at the documented ports.

11. Conclusion

The Article Platform is now fully containerized: two purpose-built multi-stage images for the backend and frontend, an unmodified official image for the database, and a single Docker Compose file that wires all three together with correct startup ordering, an internal network, a persistent volume, and externally-supplied secrets. No application source file required modification to support this — the existing use of Spring Boot's environment-variable overrides and Vite's build-time variables meant the containerization layer could be added purely additively, which was a deliberate design goal throughout this phase rather than an accident.